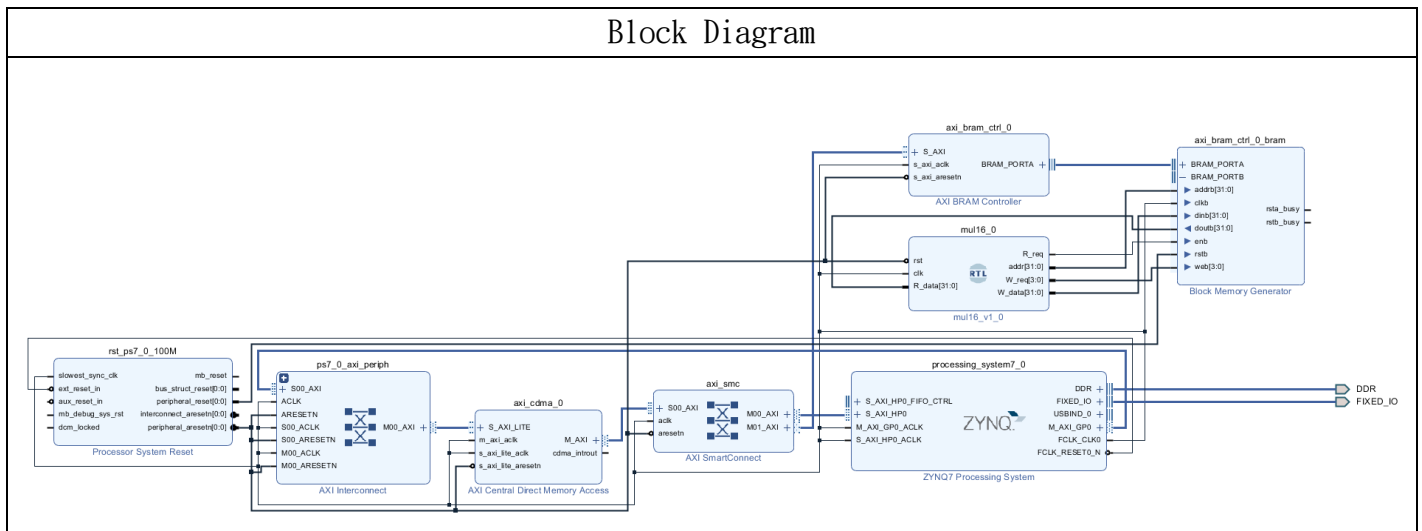


HOMework 8

Student ID: E34106010

Name: 黃偉峰

Block design Screenshot:



1. ZYNQ7:

- 使用當中的 ARM 處理器，在 Linux 中執行 Python。
- 提供 DDR 記憶體並透過 HP port 與 CDMA 傳輸資料。

2. AXI CDMA:

- 負責從 DDR 傳輸資料至 BRAM，以及將運算結果從 BRAM 傳回 DDR。
- 只須設定內部 Register 即可搬運資料，無需 CPU 介入傳輸。

3. AXI BRAM Controller + Block Memory Generator:

- 接收從 DDR 透過 CDMA 搬運過來的 operand，並傳輸給 mul16 IP 使用。
- 計算完成後也將結果寫回 BRAM。

4. mul16_v1_0:

- 讀取 BRAM 中的兩筆 operand，自動執行乘法並將結果寫入 BRAM 的特定位置。
- 在 BRAM 資料被寫入後自動觸發運算。

Jupyter python code:

Python Code

```
cdma = MMIO(0x7E200000, 0x10000) # Range = 0x10000 = 2^16 = 64KB  
ddr = MMIO(0x30000000, 0x4000) # Range = 0x4000 = 2^14 = 16KB  
bram = MMIO(0xC0000000, 0x4000) # Range = 0x4000 = 2^14 = 16KB
```

根據在 Vivado 的 Address Editor 中所設定的 base address 與對應的 Range，在 Python 中透過 MMIO 物件進行對應的 Memory Map

```
# STEP 1: ARM cpu write data to DDR  
a = 60  
b = 20  
ddr.write(0x00, a) # Write a to 0x30000000  
ddr.write(0x04, b) # Write b to 0x30000004
```

將兩個乘法運算的 operand a 與 b 分別寫入 DDR 記憶體中：

- a = 60 被寫入至 0x30000000
- b = 20 被寫入至 0x30000004

```
# STEP 2: CDMA transfers data from DDR to BRAM  
cdma.write(0x00, 0x04) # Start CDMA  
cdma.write(0x18, 0x30000000) # Source: DDR  
cdma.write(0x20, 0xC0000000) # Dest: BRAM  
cdma.write(0x28, 0x08) # BTT = 8 bytes (2 word)  
  
time.sleep(0.01) # Wait for CDMA transfer to complete
```

啟動 AXI CDMA，將 ZYNQ DDR 中的兩筆 operand 搬移至 BRAM。對應設定如下：

- cdma.write(0x00, 0x04)：啟動 CDMA
- cdma.write(0x18, 0x30000000)：設定 Source Address 為 DDR
- cdma.write(0x20, 0xC0000000)：設定 Destination Address 為 BRAM
- cdma.write(0x28, 0x08)：傳輸 8 bytes (2 個 32-bit word)

```
# STEP 3: CDMA transfers data from BRAM back to DDR (return result)
cdma.write(0x00, 0x04)           # Start CDMA transfer
cdma.write(0x18, 0xC000000C)     # Source address: BRAM
cdma.write(0x20, 0x3000000C)     # Destination address: DDR
cdma.write(0x28, 0x10)           # BTT = 16 bytes (4 words)

time.sleep(0.01)                 # Wait for CDMA transfer to complete
```

透過 AXI CDMA 將乘法結果從 BRAM 傳回 DDR。

對應設定如下：

- cdma.write(0x00, 0x04)：啟動 CDMA 傳輸
- cdma.write(0x18, 0xC000000C)：設定 Source Address 為 BRAM
- cdma.write(0x20, 0x3000000C)：設定 Destination Address 為 DDR
- cdma.write(0x28, 0x10)：設定搬運 16 bytes (4 個 32-bit word)

```
# -----
# STEP 4: Read the result from DDR (at offset 0x0C)
result = ddr.read(0x0C)
print(f"ans of {a} * {b} = {result}")
```

從 DDR 的位址 0x3000000C 讀取乘法結果。

Lesson learn

透過本次作業，我實際體驗了如何使用 CDMA，DMA 這個東西過往在計算機組織的課程當中就有學到但從來沒有實際自己使用過，這次實作讓我理解了 CDMA 是如何透過設定 source、destination 以及傳輸 Byte 數，就能達成資料從 DDR 到 BRAM 的搬運，而無需 CPU 介入整個傳輸過程，此外，我也更加熟悉了 AXI 各類 interface 的用途，例如 AXI-Lite 用於控制 IP、而 AXI HP port 則用於大量資料的搬運。本次作業最大的挑戰，我認為是一開始看到作業要求的 Block Design 架構圖時，無法馬上分辨每個 IP 模組之間是如何連接、誰控制誰、誰傳資料給誰。在實作過程中，我不只需要看懂 IP 之間的關係，還需要根據 Address Editor 中設定的記憶體對應位址，才能正確用 MMIO 在 Python 中操作這些 IP。